Adiabatic Quantum Computation of QUBO Problems

Hrishikesh Belagali

CMSE 890 Project

Abstract

Combinatorial optimization problems find applications in many fields, and are often computationally demanding to solve. Classical heuristic algorithms are commonly used to find approximate solutions, but quantum computing has been explored as a potential approach to solve these problems more efficiently. In this project, we implement an adiabatic quantum computation (AQC) algorithm to solve Quadratic Unconstrained Binary Optimization (QUBO) problems, which are a specific class of combinatorial optimization problems. We analyze the performance of the AQC algorithm by varying parameters such as total execution time, number of Trotter steps, spectral gap, and schedule functions. We compare the results with classical heuristic methods, and evaluate algorithmic behavior on the `ibm_kingston` quantum device. We find that the quadratic schedule function performs better than the linear and sinusoidal schedules, likely due to its slower evolution near the minimum spectral gap. We also find that the success probability degrades significantly on the quantum hardware compared to the simulator, and this degradation is not significantly reduced by readout error mitigation, suggesting that the dominant source of error is through decoherence and gate errors. These results provide insights into engineering AQC algorithms for practical applications, and highlight some of the challenges associated with implementing AQC on near-term quantum devices.

1 Introduction

Combinatorial optimization problems are ubiquitous across fields, including logistics, finance, machine learning, physics, and many others. These problems involve finding an optimal solution to a problem from a finite set of possible solutions, which often grows rapidly with the size of the problem. Many combinatorial optimization problems are NP-hard, meaning that there is no known polynomial-time algorithm to solve them exactly.

Existing methods for solving combinatorial optimization problems include exact algorithms such as branch-and-bound[1], cutting planes[2], dynamic programming[3], and systematic enumeration[4]. However, these methods can be computationally expensive and may not be feasible for large instances. Another class of methods are heuristic algorithms, such as simulated annealing[5], tabu search[6], genetic algorithms[7], and many others. These algorithms are often used when exact methods are too slow, but they do not guarantee optimal solutions.

Quantum computing has been explored as a potential approach to solving combinatorial optimization problems more efficiently than classical algorithms, leveraging phenomena such as superposition and entanglement. One of the earliest algorithms proposed was quantum stochastic optimization, which was a procedure based on the quantum tunnel effect to escape local minima in optimization problems[8]. It was later renamed to quantum annealing (QA) and proposed as an alternative to simulated annealing for solving combinatorial optimization problems, in which quantum fluctuations were used to escape local minima instead of thermal fluctuations[9]. It was later shown that quantum annealing demonstrated faster convergence compared to simulated annealing for certain classes of problems[10]. The experimental implementation of QA in a disordered quantum ferromagnet allowed for the algorithm to be considered as an alternative to the standard gate model, involving a designated quantum device[11].

Adiabatic quantum computation (AQC) is another optimization method which aims to leverage quantum mechanics to solve optimization problems. It relies on the adiabatic theorem of quantum mechanics, and works by encoding the solution to the problem in the ground state of a Hamiltonian. Then the system is slowly evolved from an initial Hamiltonian with a known ground state to the problem Hamiltonian. If the evolution is slow enough, the system will remain in the ground state, yielding the optimal solution to the problem.

It has been shown that non-stoquastic AQC, i.e. AQC with a non-stoquastic Hamiltonian is as powerful as the circuit model of quantum computation, and thus can solve any problem that a universal quantum computer can with atmost polynomial overhead[12].

In this project, we will take a specific class of combinatorial optimization problems, known as Quadratic Unconstrained Binary Optimization (QUBO) problems, and implement an adiabatic evolution algorithm to solve them. We will compare the results with classical heuristic methods and analyze the performance. The goal of the project is to understand the factors that influence AQC's performance and behavior.

2 Problem Formulation

2.1 Quadratic Unconstrained Binary Optimization

The Quadratic Unconstrained Binary Optimization (QUBO) problem is a combinatorial optimization problem that minimizes a quadratic objective function defined over binary variables. Formally, given a symmetric ma-

trix $\mathbf{Q} \in \mathbb{R}^{n\times n}$, the QUBO problem can be expressed as:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\in\{0,1\}^n} \mathbf{x}^\top \mathbf{Q}\mathbf{x}$$

Note that $\mathbf{Q}$ is generally assumed to be symmetric, however this is not a strict requirement since we can always symmetrize it by replacing $\mathbf{Q}$ with $\frac{\mathbf{Q}+\mathbf{Q}^\top}{2}$ without changing the optimization problem. Several combinatorial problems can be reformulated as QUBO problems, including the Max-Cut problem, Graph Coloring, the Traveling Salesman Problem, and many others[13]. In this project, we will formulate the QUBO problem as a Hamiltonian and use adiabatic evolution to find the optimal solution.

2.2 Hamiltonian Formulation

The QUBO problem can be written as

$$f(\mathbf{x}) = \sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{Q}_{ij} x_i x_j$$

First, we will map the variables from $\{0,1\}$ to $\{-1,1\}$ using the transformation

$$x_i = \frac{1-s_i}{2}$$

where $s_i \in \{-1,1\}$ is a spin variable. Then, f can be rewritten as

$$\begin{aligned} f(\mathbf{s}) &= \sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{Q}_{ij} \frac{1-s_i}{2}\frac{1-s_j}{2} \\ &= \sum_{i<j} \mathbf{J}_{ij} s_i s_j + \sum_{i=1}^{n} \mathbf{h}_i s_i + E_0 \mathbf{I} \end{aligned}$$

where J_{ij}, h_i, E_0 are derived from $\mathbf{Q}$. The full derivation is given in Appendix 5. Note that the constant does not affect the optimization, so we can ignore it. The Hamiltonian is then defined by replacing the spin variables with the corresponding Pauli-Z operators.

$$\mathcal{H} = \sum_{i<j} \mathbf{J}_{ij} Z_i Z_j + \sum_{i=1}^{n} \mathbf{h}_i Z_i$$

Now, the ground state of $\mathcal{H}$ corresponds to the optimal solution of the QUBO problem. It is found by implementing the adiabatic evolution as a time evolution operator, and measuring the final state in the computational basis.

2.3 Adiabatic Evolution

In adiabatic quantum computation, we start with an initial Hamiltonian $\mathcal{H}_0$ whose ground state is trivial to prepare. Then, we define a time-dependent Hamiltonian that interpolates between $\mathcal{H}_0$ and the problem Hamiltonian $\mathcal{H}$. The solution to the problem is obtained by evolving the system according to the time-dependent Schrodinger equation and measuring the final state. The key constraint is that the evolution must be slow enough to ensure that the system remains in its ground state throughout the process, which is guaranteed by the adiabatic theorem. The total evolution time T must satisfy

$$T \gg \frac{1}{\Delta^2}$$

where Δ is the minimum energy gap between the ground state and the first excited state of the Hamiltonian during the evolution. Here, we define the initial Hamiltonian as

$$\mathcal{H}_0 = -\sum_{i=1}^{n} X_i$$

where X_i is the Pauli-X operator acting on the i-th qubit. The ground state of $\mathcal{H}_0$ is $|+\rangle^{\otimes n}$, which is a uniform superposition of all possible states. Next, we define the problem Hamiltonian $\mathcal{H}_Q$ as derived in the previous section. Finally, we define the time-dependent Hamiltonian $\mathcal{H}_A(t)$ as

$$\mathcal{H}_A(t) = (1-s(t))\mathcal{H}_0 + s(t)\mathcal{H}_Q$$

where $s(t)$ is a monotonic function that satisfies $s(0)=0$ and $s(T)=1$.

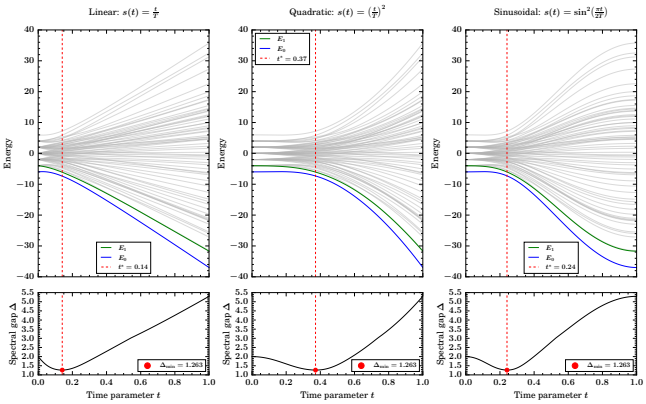

Figure 1: Evolution of the eigenvalues of the Hamiltonian $\mathcal{H}_A(t)$ during the adiabatic evolution. The ground state is shown in blue, and the first excited state is shown in green. The minimum energy gap Δ is the minimum distance between the blue and green curves. The schedule function is given by $s(t)$

The evolution is Trotterized (Section 2.4) into discrete time steps, and at each step, we apply the corresponding unitary operator to evolve the state. Note that as $\Delta t \to 0$, the Hamiltonian is effectively constant during each time step, and the evolution can be approximated by a sequence of unitary operations corresponding to the Hamiltonian at each time step.

2.4 Trotterization

The time evolution operator for a time-dependent Hamiltonian $\mathcal{H}(t)$ is given by the time-ordered exponential:

$$U(T) = \mathcal{T}\exp\left(-i\int_0^T \mathcal{H}(t)dt\right)$$

However, this operator is difficult to compute directly. Instead, we can approximate it using Trotterization, which breaks the evolution into small time steps Δt and approximates the evolution as a product of exponentials of the Hamiltonian at each time step.

Let T be the total evolution time, and let N be the number of time steps, so that $\Delta t = \frac{T}{N}$ by construction. The first order Trotterization is given by

$$e^{-i(A+B)\Delta t} \approx e^{-iA\Delta t} e^{-iB\Delta t}$$

where A and B are the two parts of the Hamiltonian. Higher order Trotterization can be achieved by using more complex sequences of exponentials, such as the second order Trotterization. The local truncation error is $O(\Delta t^2)$ for the first order Trotterization and $O(\Delta t^3)$ for the second order Trotterization, and the global error is $\mathcal{O}(\frac{T^2}{N})$ and $\mathcal{O}(\frac{T^3}{N^2})$ respectively, assuming that A and B do not have explicit time dependence.

For a time dependent Hamiltonian, we approximate that the Hamiltonian is roughly constant during each time step, and apply the Trotterization to the Hamiltonian at each time t.

2.5 Implementation Details

The implementation of AQC was done using `qiskit`[14] 2.3.1. The Trotterized evolution was implemented using the `PauliEvolutionGate`, `LieTrotter` and `SuzukiTrotter` classes from `qiskit`. The algorithm was tested on the `Aer` simulator, and run on IBM's `ibm_kingston` device to analyze performance in a noisy environment. The `mthree` package was used for readout error mitigation. Calculations were done using `NumPy`[15]. The code is available at this repository.

3 Results

We systematically varied parameters involved in the AQC algorithm in order to analyze their effects on the performance. We compared the results with simulated annealing, and analyzed the performance on a noisy quantum device. The details of the results are given in the following subsections.

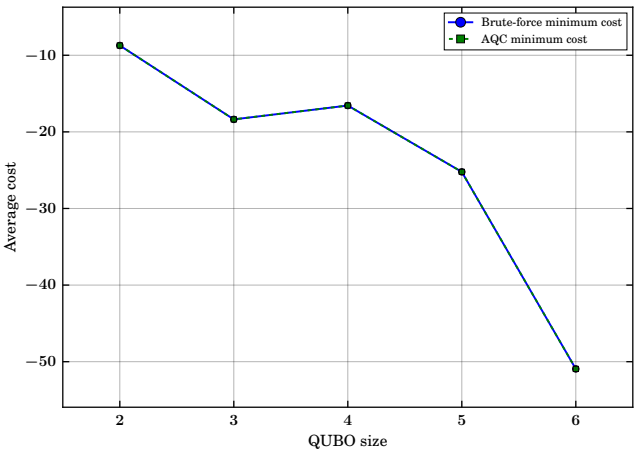

Figure 2: Minimum state energy found by AQC and simulated annealing for 100 random QUBO instances on n qubits. The AQC algorithm was run with a total evolution time of $T = 10.0$, 100 Trotter steps, and a linear schedule.

3.1 Total Evolution Time

We varied the total evolution time T keeping the number of Trotter steps fixed, and analyzed the effect on the success probability p of finding the optimal solution. 100 random QUBO instances were generated for each value of T, and the AQC algorithm was run on each instance to compute the success probability. The Trotter order was fixed to 2, the number of shots was fixed to 4096, and the number of iterations was fixed to 50. The success probability was compared to simulated annealing on the same QUBO instances. The results are shown in Figure 3.

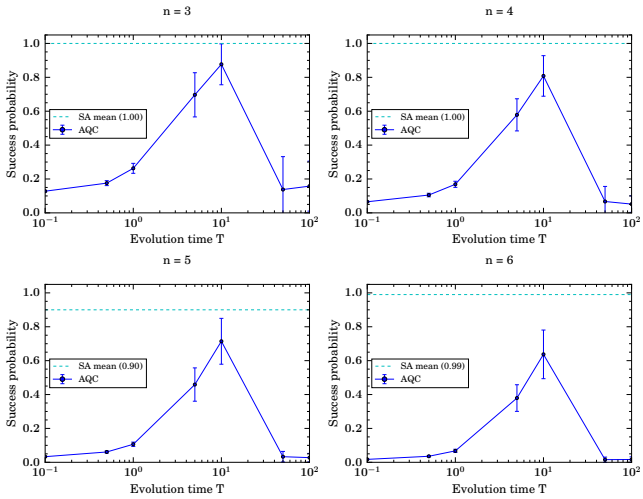

Figure 3: Effect of total evolution time on the success probability for QUBO instances of n qubits.

The results show that the success probability peaks at a specific value of T for which the evolution is large enough to allow the system to remain in the ground state, but not too large to accumulate significant Trotterization error. The success probability is also lower than simulated annealing across the range of T values, likely due to adiabatic and Trotterization errors.

3.2 Trotterization Steps

We varied the number of Trotter steps N while keeping the total evolution time T fixed, and analyzed the effect on the success probability. The total time T was fixed at 10.0, and all other parameters were kept the same as in Section 3.1. The results are shown in Figure 4.

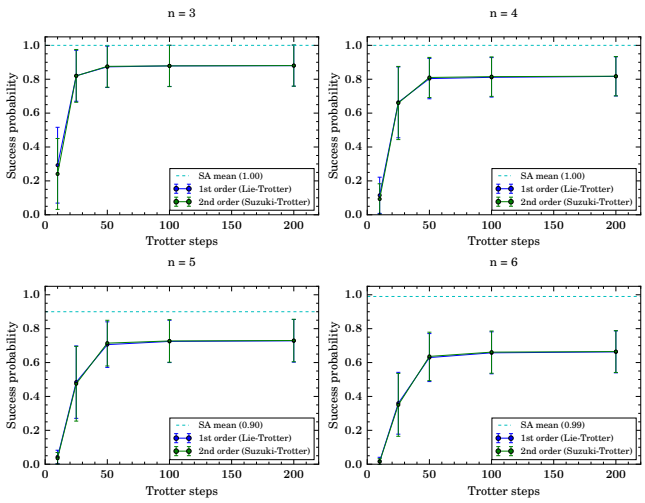

Figure 4: Effect of number of Trotter steps on the success probability for QUBO instances of n qubits.

The results show that the success probability increases with the number of Trotter steps as expected. The Trotter error decreases with increasing N, leading to a more accurate approximation of the time evolution operator. After a certain point, the success probability flattens out, likely due to the fact that the Trotter error becomes negligible, and the success probability is limited by other

factors such as adiabatic error. Additionally, the success probability is virtually the same for the two Trotterization orders, suggesting that the Trotter error is not the dominant source of error in this regime.

3.3 Spectral Gap

We compute the spectral gap Δ for each QUBO instance, and analyze its correlation with the success probability p. The spectral gap is computed by diagonalizing the Hamiltonian $\mathcal{H}_A(t)$ at each time step, and finding the minimum energy gap between the ground state and the first excited state. The results are shown in Figure 5.

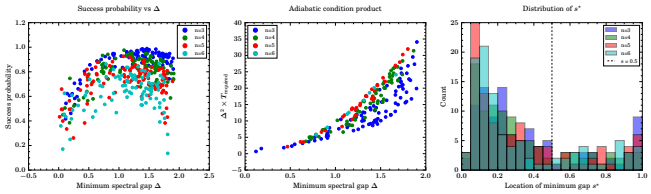

Figure 5: Correlation between spectral gap Δ and success probability p for QUBO instances of n qubits.

The success probability is positively correlated with the spectral gap, as expected from the adiabatic theorem. Instances with larger spectral gaps have higher success probabilities, since they have a lower threshold evolution time T required to satisfy the adiabatic condition. In general, the minimum spectral gap appears to occur in the first half of the evolution (Figure 5). This result sheds light on how to engineer the schedule function $s(t)$, in order to slow down the evolution during the period of the minimum spectral gap, which preserves the non-degeneracy of the eigenvalues and allows the system to remain in the ground state throughout the evolution.

3.4 Schedule Functions

We analyze the effect of different schedule functions $s(t)$ on the success probability. We compare the following schedules:

$$s(t) = \frac{t}{T} \qquad \textit{(linear)} \qquad (1)$$

$$s(t) = \left(\frac{t}{T}\right)^2 \qquad \textit{(quadratic)} \qquad (2)$$

$$s(t) = \sin^2\left(\frac{\pi t}{2T}\right) \qquad \textit{(sinusoidal)} \qquad (3)$$

The results are shown in Figure 6.

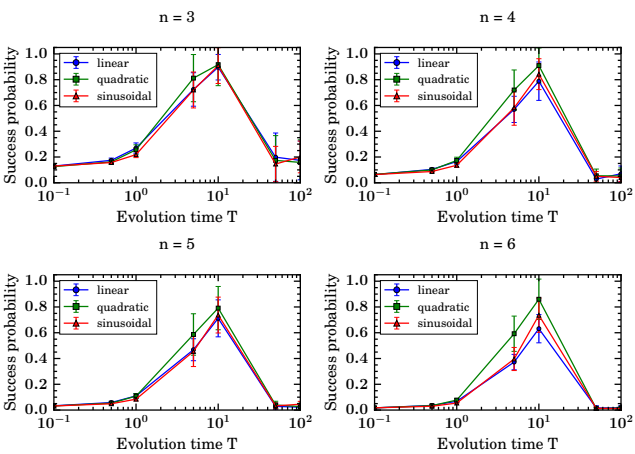

Figure 6: Effect of different schedule functions on the success probability for QUBO instances of n qubits.

The success probability has a non-trivial dependence on the schedule function. The values are roughly similar for the linear and sinusoidal schedules, while the quadratic schedule has a higher success probability for the larger QUBO instances. The minimum spectral gap is identical for all schedules (Figure 1), because it is a property of the Hamiltonian $\mathcal{H}_A(t)$ and therefore independent of the schedule function. However, the schedule function affects the success probability by changing the rate of evolution. Particularly, the quadratic schedule evolves more slowly near the minimum spectral gap, which permits the system to remain in the ground state.

3.5 Quantum Hardware Performance

We run the AQC algorithm on IBM's `ibm_kingston` device, and compare the results with the `Aer` simulator. The results are shown in Figure 7.

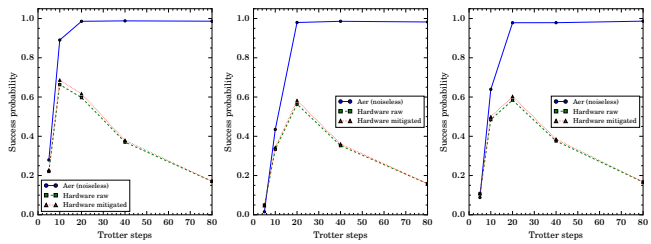

Figure 7: Comparison of success probability on `ibm_kingston` device and `Aer` simulator for QUBO instances of 6 qubits. Readout error mitigation was applied to the hardware results using `mthree`.

The success probability on the hardware is significantly lower than the simulator, likely due to the presence of noise and errors in the quantum device. The results show that the performance degrades as the number of qubits increases, which is expected due to higher noise and error rates for larger circuits. Interestingly, the success rate does not significantly change with or without the readout error mitigation, suggesting that the dominant source of error is not through measurements, but rather through gate errors and decoherence during the evolution.

4 Discussion

In this project, we implemented an adiabatic quantum computation algorithm to solve QUBO problems of 3 to 6 qubits, and systematically varied the evolution time, Trotter steps, spectral gap, and schedule function to assess their impact on success probability. We observed that the evolution time has a critical value that maximizes the success probability, prior to which the evolution is too fast to satisfy the adiabatic condition, and after which the Trotterization error becomes significant. We also observed that the spectral gap is positively correlated with the success probability, and that the quadratic schedule performs better than the linear and sinusoidal schedules due to its slower evolution near the minimum spectral gap. Finally, we found that the performance on the quantum hardware is significantly worse than the noiseless `Aer` simulator. The dominant source of error appears to be through gate errors and decoherence rather

than readout errors, which is why readout error mitigation does little to improve the success probability.

We chose to analyze the performance of AQC over other optimization algorithms due to its inherent differences, namely the ability to implement it on a designated quantum device that lies outside the standard gate model. Trotterization was selected as the method for approximating the time evolution due to its simplicity. Some room for future work includes larger scale experiments on more than 6 qubits, although this is difficult due to queue times and noise on current devices. Additionally, it would be interesting to analyze the performance of AQC on other classes of combinatorial optimization problems, such as Max-Cut or Graph Coloring.

5 Contributions

The work in this project was done by Hrishikesh Belagali.

Appendix

The general code structure for the AQC algorithm is as follows:

Require: QUBO matrix $\mathbf{Q} \in \mathbb{R}^{n\times n}$, total evolution time T, number of iterations N, Trotter order k, schedule function $s(t)$

Ensure: Measurement outcome $\mathbf{x}^* \in \{0,1\}^n$

```
Compute Ising coefficients J_ij, h_i, E_0 from Q          ▷ Appendix derivation
H_0 ← −Σ_{i=1}^n X_i                                      ▷ Driver Hamiltonian
H_Q ← Σ_{i<j} J_ij Z_i Z_j + Σ_{i=1}^n h_i Z_i            ▷ Problem Hamiltonian
Δt ← T/N
|ψ⟩ ← |+⟩^{⊗n}                                           ▷ Apply H gate to each qubit
for m = 0, 1, ..., N − 1 do
    t_m ← m/(N − 1)
    s_m ← s(t_m)                                          ▷ Evaluate schedule function
    H_A ← (1 − s_m) H_0 + s_m H_Q                         ▷ Adiabatic Hamiltonian
    if k = 1 then                                         ▷ Lie–Trotter
        U_m ← e^{−i (1−s_m) H_0 Δt} e^{−i s_m H_Q Δt}
    else if k = 2 then                                    ▷ Suzuki–Trotter
        U_m ← e^{−i (1−s_m) H_0 Δt/2} e^{−i s_m H_Q Δt} e^{−i (1−s_m) H_0 Δt/2}
    end if
    |ψ⟩ ← U_m |ψ⟩
end for
Measure |ψ⟩ in the computational basis → x*
return x*
```

The complete code for this project is available at this repository.

Derivation of Hamiltonian from QUBO

Let $\mathbf{Q} \in \mathbb{R}^{n\times n}$ be the matrix defining the QUBO problem, and let $\mathbf{x} \in \{0,1\}^n$ be a binary vector. Without loss of generality, we can assume that the QUBO matrix $\mathbf{Q}$ is symmetric. The objective function is given by

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q}\mathbf{x} = \sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{Q}_{ij} x_i x_j$$

We substitute the spin variable transformation $x_i = \frac{1-s_i}{2}$, where $s_i \in \{-1, 1\}$.

$$\begin{aligned} f(\mathbf{s}) &= \sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{Q}_{ij}\left(\frac{1-s_i}{2}\right)\left(\frac{1-s_j}{2}\right) \\ &= \frac{1}{4}\sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{Q}_{ij}(1-s_i)(1-s_j) \\ &= \frac{1}{4}\sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{Q}_{ij}(1-s_i-s_j+s_is_j) \\ &= \frac{1}{2}\sum_{i=1}^{n}\frac{\mathbf{Q}_{ii}}{2} + \frac{1}{4}\sum_{i<j}\mathbf{Q}_{ij} - \sum_{i=1}^{n}\frac{\mathbf{Q}_{ii}}{2}s_i \\ &+ \frac{1}{4}\sum_{i<j}\mathbf{Q}_{ij}(s_i+s_j) + \frac{1}{4}\sum_{i<j}\mathbf{Q}_{ij}s_is_j \end{aligned}$$

Now, we can define the Hamiltonian by replacing the spin variables with the corresponding Pauli Z operators:

$$\mathcal{H}_Q = E_0 I + \sum_{i=1}^{n}\mathbf{h}_i Z_i + \sum_{i<j}\mathbf{J}_{ij} Z_i Z_j$$

where

$$\mathbf{h}_i = -\frac{\mathbf{Q}_{ii}}{2} - \sum_{i\neq j}\frac{\mathbf{Q}_{ij}}{4}$$

$$\mathbf{J}_{ij} = \frac{\mathbf{Q}_{ij}}{4}$$

$$E_0 = \frac{1}{2}\sum_{i=1}^{n}\frac{\mathbf{Q}_{ii}}{2} + \frac{1}{4}\sum_{i\neq j}\mathbf{Q}_{ij}$$

References

[1] An Automatic Method for Solving Discrete Programming Problems | Springer Nature Link. https://link.springer.com/chapter/10.1007/978-3-540-68279-0_5.

[2] On Cutting Planes. In *Annals of Discrete Mathematics*, volume 9, pages 291–296. Elsevier, January 1980.

[3] R. Bellman, R.E. Bellman, and Rand Corporation. *Dynamic Programming*. Rand Corporation Research Study. Princeton University Press, 1957.

[4] Egon Balas. An Additive Algorithm for Solving Linear Programs with Zero-One Variables. *Oper. Res.*, 13(4):517–546, August 1965.

[5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by Simulated Annealing. *Science*, 220(4598):671–680, May 1983.

[6] Fred Glover and Manuel Laguna. Tabu search. In *Modern Heuristic Techniques for Combinatorial Problems*, pages 70–150. John Wiley & Sons, Inc., USA, May 1993.

[7] Kumara Sastry, David Goldberg, and Graham Kendall. Genetic Algorithms. In Edmund K. Burke and Graham Kendall, editors, *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques*, pages 97–125. Springer US, Boston, MA, 2005.

[8] B. Apolloni, C. Carvalho, and D. de Falco. Quantum stochastic optimization. *Stochastic Processes and their Applications*, 33(2):233–244, December 1989.

[9] (PDF) A numerical implementation of quantum annealing. In *ResearchGate*.

[10] Tadashi Kadowaki and Hidetoshi Nishimori. Quantum annealing in the transverse Ising model. *Physical Review E*, 58(5):5355–5363, November 1998.

[11] Quantum Annealing of a Disordered Magnet | Science. https://www.science.org/doi/10.1126/science.284.5415.779.

[12] Adiabatic quantum computation | Rev. Mod. Phys. https://journals.aps.org/rmp/abstract/10.1103/RevModPhys.90.015002.

[13] Fred Glover, Gary Kochenberger, and Yu Du. A Tutorial on Formulating and Using QUBO Models, November 2019.

[14] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with Qiskit, June 2024.

[15] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.